Multi-Agent Architecture Overview

iosys AG — Personal Agent Ecosystem — 2026-02
6 repos · 59 skills · 7 MCP servers · ~80 solution docs

1. Architecture: Hub-and-Spoke

`personal-agent` is the orchestration hub. All other repos are specialized spokes that share conventions, file paths, and data flows — but operate independently in their own Claude Code sessions.

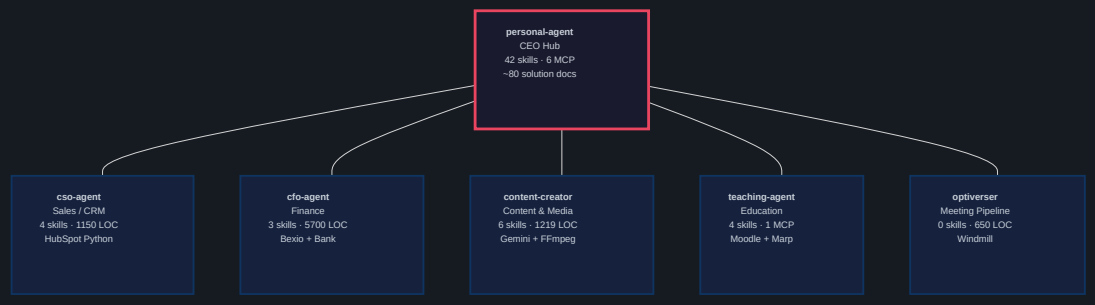

Fig 1 — Hub-and-spoke topology with key stats per repo

2. Per-Repo Summary

Repo	Purpose	Skills	LOC	MCP	Maturity	Self-Audit
personal-agent	CEO hub: TODOs, calendar, email, billing, meetings	42	—	6	Production	69%
cso-agent	Sales: HubSpot pipeline, forecasting, events	4	1,150	0	MVP	59%
cfo-agent	Finance: cashflow, bank imports, receipts, invoicing	3	5,700	0	Production	43%
content-creator	LinkedIn posts, case studies, video collage, images	6	1,219	0	Production	—
teaching-agent	Moodle LMS, Marp lecture slides, course validation	4	—	1	Operational	—
optiverser	Meeting transcript pipeline (Windmill → Teams)	0	650	0	Operational	—

3. Cross-Repo Dependency Map

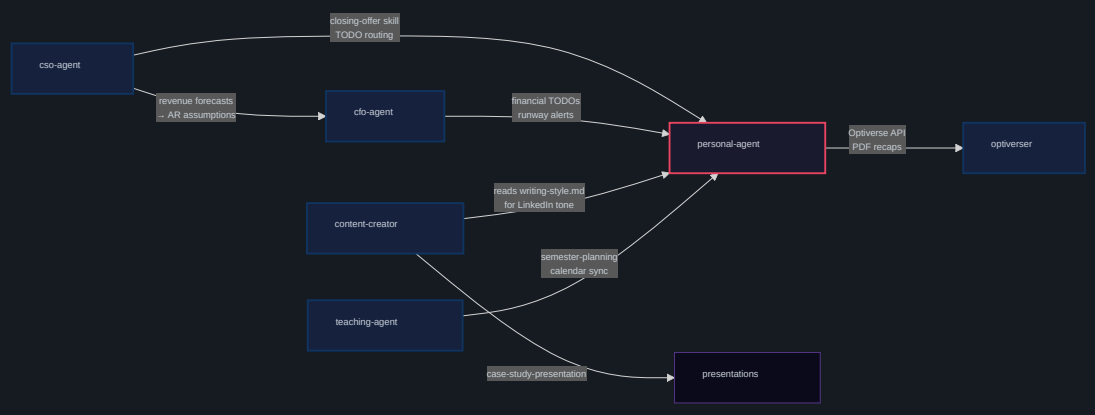

Fig 2 — Directed data and skill flows between repos

From	To	What Flows
cso-agent	personal-agent	`/create-closing-offer` skill, TODO routing to Eisenhower Matrix
cso-agent	cfo-agent	Revenue forecasts inform 13-week cashflow AR assumptions
cfo-agent	personal-agent	Financial TODOs, runway alerts
content-creator	personal-agent	Reads `persona/writing-style.md` for LinkedIn tone
content-creator	presentations	`/case-study-presentation` skill reference
teaching-agent	personal-agent	`/semester-planning` for calendar integration
personal-agent	optiverser	Same Optiverse API, different audience

4. Data Flow: Email → Action Pipeline

The primary intake funnel. Flagged Outlook emails are triaged by `/organize-inbox` into Linear tickets (customer/project work) or TODO items (CEO topics), using rule-based routing defined in `rules.md`.

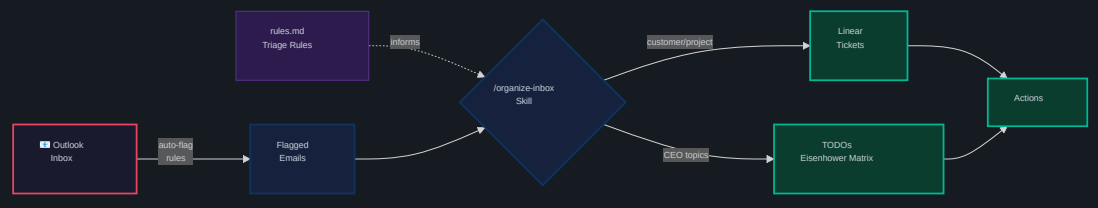

Fig 3 — Email intake and triage pipeline

5. Shared Filesystem Map

The repos don't share code — they share *paths*. Multiple agents read and write to the same directories under `~/Documents/iosys/`, with external APIs accessed by multiple repos in parallel.

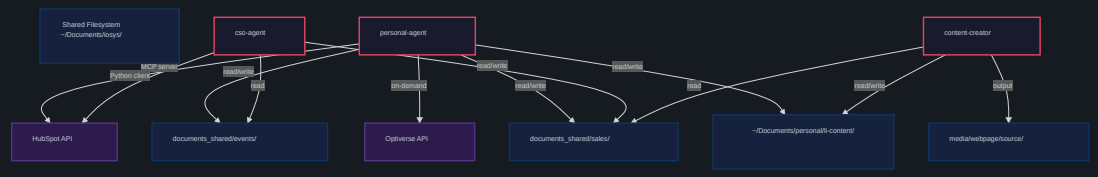

Fig 4 — Filesystem and API access patterns across repos

6. Most Impactful Skills

personal-agent (top 10 of 42)

Skill	Impact
`/billing-sync`	2,036 LOC script; monthly invoicing pipeline (Toggl → Bexio)
`/organize-inbox`	Email → Linear ticket triage; primary intake funnel
`/weekly-prep`	Monday dashboard: calendar + TODOs + Linear
`/send-meeting-recap`	Optiverse → PDF → email draft for external stakeholders
`/organize-meeting-transcripts`	PRDs + relationship maps from customer calls
`/strategic-planner`	Strategy-as-YAML with testable hypotheses and confidence levels
`/create-closing-offer`	End-to-end sales closing package (Marp + offer + email)
`/prep-meeting`	Company research + attendee profiles before calls
`/create-outlook-event`	Graph API calendar with dual-calendar pattern
`/create-linear-ticket`	URL/context → structured Linear ticket

cso-agent

Skill	Impact
`/forecast`	Weighted pipeline revenue projections with confidence intervals
`/pipeline`	Stale deal detection, concentration risk, health scoring
`/event-postprocessing`	Business card OCR → HubSpot contact upsert

cfo-agent

Skill	Impact
`/cashflow`	13-week rolling forecast with runway calculation
`/bank-import`	CAMT.053/MT940 parsing into SQLite with dedup
`/receipt-check`	Bexio expense vs receipt reconciliation

content-creator

Skill	Impact
`/linkedin-post-creator`	Bilingual posts with distinct DE/EN voice, style-calibrated
`picture-generator`	Gemini API with model fallback chain, brand-consistent imagery
`video-collage`	FFmpeg-based clip assembly for LinkedIn/YouTube (722 LOC)

teaching-agent

Skill	Impact
`/reverse-eng-slides`	Story arc extraction → Marp slides with overflow detection
`/prep-lecture`	Pre-lecture briefing from Moodle enrollment + submissions

7. Emergent Primitives

Eleven patterns that crystallized organically while building the system.

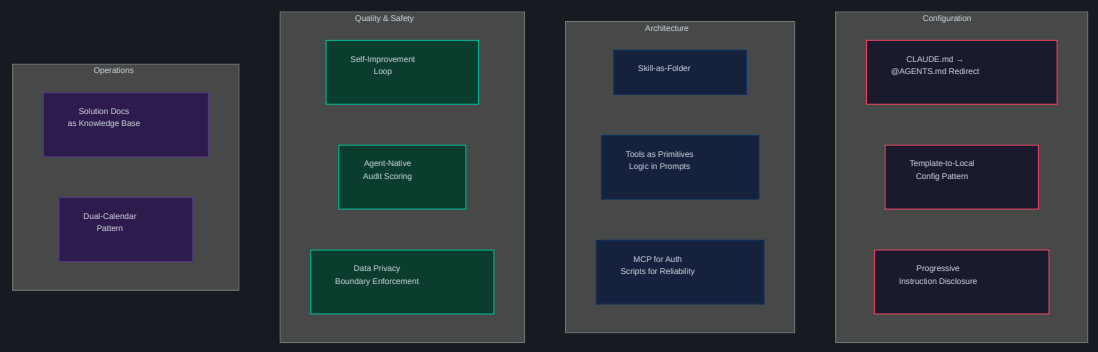

Fig 5 — Primitives taxonomy by category

1. CLAUDE.md → @AGENTS.md Redirect
Every repo uses a one-line `CLAUDE.md` pointing to `AGENTS.md`. Separates the Claude Code hook from the actual instructions.

2. Template-to-Local Config
`config.template.yaml` (committed) → `config.local.yaml` (gitignored). Same for `.env.template` → `.env`. Secrets never leak while keeping structure documented.

3. Skill-as-Folder
`.claude/skills/{name}/SKILL.md` with optional `scripts/`, `templates/`, `*.yaml`. Single-purpose, self-documenting, composable.

4. Tools as Pure Primitives, Logic in Prompts
Data-access layers have zero business logic. Forecasting, health scoring, confidence intervals — all in SKILL.md files. The agent reasons; code just fetches.

5. Solution Docs as Knowledge Base
`docs/solutions/{category}/{slug}.md` — ~80 files. Each: problem → investigation → root cause → solution → gotchas.

6. MCP for Auth, Scripts for Reliability
MCP servers handle authentication; actual operations use direct API calls (Python scripts). Token reuse pattern enables scripts to piggyback on MCP auth.

7. Self-Improvement Loop
Use → Observe friction → Refine primitive → Document in `tasks/lessons.md`. After every correction, capture the lesson.

8. Agent-Native Audit with Scoring
Formal self-audits scoring 8 principles: Action Parity, CRUD Completeness, Context Injection, Prompt-Native, Tools as Primitives, Shared Workspace, UI Integration, Capability Discovery.

9. Data Privacy Boundary Enforcement
Three-tier: *Committed* (anonymized), *Gitignored* (customer data), *Never leave machine* (financial).

10. Dual-Calendar Pattern
Business events go to both Outlook (Graph API) and Nextcloud B&T (CalDAV). Availability checks query both.

11. Progressive Instruction Disclosure
`AGENTS.md` (skim) → `instructions/*.md` (read) → `docs/solutions/*` (dive deep). Three layers for context window efficiency.

8. Key Tensions / Tech Debt

1. Dual HubSpot access
personal-agent (MCP) and cso-agent (Python client) hit the same API without rate-limit coordination.

2. Optiverser library/flow divergence
Production-grade library scripts exist but aren't wired into the deployed flow (missing PII redaction, dedup, SSRF protection).

3. Venv inconsistency
personal-agent: `/tmp/` shared venv. cfo-agent: project-local `.venv` with `uv`. content-creator: per-skill venvs.

4. Content-creator dual skill dirs
`skills/` (legacy) vs `.claude/skills/` (slash commands) — some duplicated.

5. Cross-agent integration is conceptual
cso → cfo revenue forecast flow, cso → personal-agent TODO routing — documented but not automated.

9. Maturity Spectrum

Repo	Maturity	Skills	Tests	Solution Docs	MCP	Self-Audit
personal-agent	Production	42	1 regression	~80	6	69%
cfo-agent	Production	3	2,203 LOC (6 files)	6	0	43%
cso-agent	MVP	4	0	1	0	59%
content-creator	Production	6	0	9	0	—
teaching-agent	Operational	4	0	10	1	—
optiverser	Operational	0	0	1 plan	0	—

Generated 2026-02 · iosys AG · Multi-Agent Architecture Reference v1.0